NUMPY

DATA VISUALIZATION :the graphical representation of information using elements like charts, graphs, and maps

```
!pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
import numpy as np
```

```
print(np.__version__)
```

```
2.0.2
```

```
a=np.array([1,2,3,4])
print(a)
```

```
[1 2 3 4]
```

```
b = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])
print(b)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
c = np.array([
    [
        [1,2,3,4],
        [5,6,7,8],
        [9,10,11,12]
    ],
    [
        [13,14,15,16],
        [17,18,19,20],
        [21,22,23,24]
    ]
])

print(c)
```

```
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
```

```
def greet():
  """
  this is for testing doc string
  """
  print("testing function")
print(greet())
```

```
testing function
None
```

```
print(greet.__doc__)
```

```

  this is for testing doc string

```

```
a=np.ones([1,2])
b=np.zeros([2,2])
c=np.full([2,2],7)
e=np.eye(4)
print(a,b,c,e)
```

```
[[1. 1.]] [[0. 0.]
 [0. 0.]] [[7 7]
```

```
 [7 7]] [[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

```
f=np.arange(2,10,3)
print(f)
```

```
[2 5 8]
```

```
np.linspace(2,4,6)
```

```
array([2. , 2.4, 2.8, 3.2, 3.6, 4. ])
```

```
np.random.randn(3,3)
```

```
array([[ 0.59117622, -0.08972808,  0.54470918],
       [-0.13998358,  0.00750653, -1.09890895],
       [ 0.30276732, -0.78401231, -1.12003033]])
```

```
np.random.rand(9,3)
```

```
array([[0.7915092 , 0.27959894, 0.19402071],
       [0.76010787, 0.47490708, 0.07428636],
       [0.66363684, 0.00658813, 0.79344711],
       [0.62557931, 0.078656  , 0.56545451],
       [0.16244262, 0.53908508, 0.24146961],
       [0.04588428, 0.96447606, 0.64009704],
       [0.97233473, 0.36333891, 0.18934154],
       [0.16856852, 0.74593979, 0.58936571],
       [0.7171477 , 0.21167654, 0.56791012]])
```

```
np.random.randint(1,5,size=(4,4))
```

```
array([[3, 1, 4, 3],
       [2, 4, 3, 3],
       [1, 3, 4, 2],
       [1, 4, 2, 3]])
```

```
np.empty((4,6))
```

```
array([[4.94e-324, 9.88e-324, 1.48e-323, 1.98e-323, 2.47e-323, 2.96e-323],
       [3.46e-323, 3.95e-323, 4.45e-323, 4.94e-323, 5.43e-323, 5.93e-323],
       [6.42e-323, 6.92e-323, 7.41e-323, 7.91e-323, 8.40e-323, 8.89e-323],
       [9.39e-323, 9.88e-323, 1.04e-322, 1.09e-322, 1.14e-322, 1.19e-322]])
```

Array arttibute

```
arr = np.array([["a","b","c","d","e"],
                ["1","2","3","4","5"]])
print(arr.shape)
print(arr.ndim)
print(arr.size)
print(arr.dtype)
print(arr.itemsize)
print(arr.nbytes)
```

```
(2, 5)
2
10
<U1
4
40
```

```
a=np.array([10,20,30,40,50])
print(a[0])
print(a[2])
print(a[-1])
print(a[1:4])
print(a[:3])
print(a[::2])
print(a[::-1])
```

```
10
30
50
[20 30 40]
[10 20 30]
```

```
[10 30 50]
[50 40 30 20 10]
```

```
arr=np.array([1,2,3,4,5,56,78])
```

```
print(arr[::5])
```

```
[ 1 56]
```

```
m=np.array([[15,24,23],
            [47,52,69],
            [76,83,96]])
print(m[0,0])
print(m[1,2])
print(m[0,:])
print(m[:,1])
print(m[0:2,1:3])
```

```
15
69
[15 24 23]
[24 52 83]
[[24 23]
 [52 69]]
```

```
a=np.array([10,252,82,42,17])
mask=a>15
print(mask)
print(a[mask>15])
print(a[[0,3]])
```

```
[False  True  True  True  True]
[]
[10 42]
```

array operation

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a ** 2)
print(a + 10)
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
[1 4 9]
[11 12 13]
```

```
a = np.array([1.2, 4.0, 9.0, 16.0])
np.floor(a)
```

```
array([ 1.,  4.,  9., 16.])
```

```
a = np.array([1.0, 4.0, 9.0, 16.0])
np.ceil(a)
np.round(a, 2)
```

```
array([ 1.,  4.,  9., 16.])
```

```
a = np.array([1.0, 4.0, 9.0, 16.0])
np.round(a, 2)
```

```
array([ 1.,  4.,  9., 16.])
```

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
print(sum(m))
```

```
[5 7 9]
```

```
 #1
import numpy as np

arr = np.arange(2, 21, 2)
print(arr)
```

```
[ 2  4  6  8 10 12 14 16 18 20]
```

```
#2
import numpy as np

arr = np.arange(25)
matrix = arr.reshape(5, 5)

identity = np.eye(5)
print(identity)
```

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

```
#3
import numpy as np

arr = np.array([3, 7, 1, 9, 2, 5, 8, 4])

max_val = np.max(arr)
index = np.argmax(arr)

print("Max:", max_val)
print("Index:", index)
```

```
Max: 9
Index: 3
```

```
#4
import numpy as np

arr = np.arange(24)
reshaped = arr.reshape(2, 3, 4)

print(reshaped)
```

```
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```

```
#5
import numpy as np

m = np.array([[4,9,2],[7,1,5],[3,8,6]])

mask = m > 5
result = m[mask]

print(result)
```

```
[9 7 8 6]
```

```
#6
import numpy as np

A = np.random.rand(3,3)
B = np.random.rand(3,3)

dot_product = np.dot(A, B)
trace_val = np.trace(dot_product)

print("Dot Product:\n", dot_product)
print("Trace:", trace_val)
```

```
Dot Product:
 [[0.60243199 0.31747074 0.66018053]
 [0.0594823  0.11876974 0.5995204 ]
```

```
 [0.05832842 0.15723848 0.73647051]]
Trace: 1.4576722460079994
```

```
#7
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

normalized = (arr - arr.min()) / (arr.max() - arr.min())

print(normalized)
```

```
[0.   0.25 0.5  0.75 1.  ]
```

```
#8
import numpy as np

a = np.array([1,2,3])
b = np.array([4,5,6])

v_stack = np.vstack((a, b))
h_stack = np.hstack((a, b))

print("Vertical:\n", v_stack)
print("Horizontal:\n", h_stack)
```

```
Vertical:
 [[1 2 3]
 [4 5 6]]
Horizontal:
 [1 2 3 4 5 6]
```